Advanced Features Deployment Examples

개요

이 문서는 AutoNAT, DCUtR, Relay를 통합한 고급 P2P 애플리케이션의 실제 배포 시나리오와 구현 예제를 제공합니다. 다양한 네트워크 환경과 사용 사례에 맞는 배포 전략을 다룹니다.

배포 시나리오

1. 분산 게임 네트워크

아키텍처

```
┌─────────────────┐    ┌─────────────────┐    ┌─────────────────┐
│    Game Client  │    │    Relay Server │    │   Game Client   │
│    (Player A)   │◄──►│   (Public IP)   │◄──►│    (Player B)   │
│     NAT 뒤      │    │                 │    │      NAT 뒤     │
└─────────────────┘    └─────────────────┘    └─────────────────┘
         │                      │                      │
         └──────────────────────┼──────────────────────┘
                                │
                        ┌─────────────────┐
                        │   Game Server   │
                        │  (Matchmaking)  │
```

└──────────────────┘

구현 예제

```
// game_network.rs
use libp2p::{autonat, dcutr, identify, ping, relay, PeerId, Multiaddr};
use std::{collections::HashMap, time::Duration};
use tokio::sync::mpsc;

/// 게임 네트워크 노드
pub  struct GameNetworkNode {
p2p_node: AdvancedP2PNode,
game_state: GameState,
player_connections: HashMap<PeerId, PlayerConnection>,
message_sender: mpsc::UnboundedSender<GameMessage>,
message_receiver: mpsc::UnboundedReceiver<GameMessage>,
}

#[derive(Debug, Clone)]
pub struct GameState {
room_id: String,
players: Vec<PeerId>,
is_host: bool,
game_started: bool,
```

```
}

#[derive(Debug)]
pub struct PlayerConnection {
peer_id: PeerId,
connection_type: ConnectionType,
latency: Duration,
last_seen: std::time::Instant,
}

#[derive(Debug, Clone)]
pub enum ConnectionType {
Direct,
Relayed,
Upgrading, // DCUtR in progress
}

#[derive(Debug, Clone)]
pub enum GameMessage {
PlayerJoined(PeerId),
PlayerLeft(PeerId),
GameData(Vec<u8>),
LatencyTest(std::time::Instant),
```

```
}

impl GameNetworkNode {
pub async fn new_game_client(relay_servers: Vec<(PeerId, Multiaddr)>) -> Result<
let config = NodeConfig {
enable_relay_server: false,
relay_servers,
aggressive_hole_punching: true, // 게임은 낮은 지연시간 중요
connection_timeout: Duration::from_secs(10), // 빠른 연결 필요
autonat_config: autonat::Config {
retry_interval: Duration::from_secs(15),
refresh_interval: Duration::from_secs(120),
confidence_max: 2, // 빠른 상태 변경
..Default::default()
},
};

let p2p_node = AdvancedP2PNode::new(config).await?;
let (tx, rx) = mpsc::unbounded_channel();

Ok(Self {
p2p_node,
game_state: GameState {
```

```
room_id: String::new(),
players: Vec::new(),
is_host: false,
game_started: false,
},
player_connections: HashMap::new(),
message_sender: tx,
message_receiver: rx,
})
}

pub async fn join_game_room(&mut self, room_id: String, host_peer_id: PeerId) ->
info!("Joining game room: {} with host: {}", room_id, host_peer_id);

// 호스트에 연결 시도
self.p2p_node.connect_to_peer(host_peer_id, vec![]).await?;

self.game_state.room_id = room_id;
self.game_state.players.push(host_peer_id);

// 연결 품질 모니터링 시작
self.start_connection_monitoring().await;
```

```
Ok(())
}

pub async fn create_game_room(&mut self, room_id: String) -> Result<(), Box<dyn
info!("Creating game room: {}", room_id);

self.game_state.room_id = room_id;
self.game_state.is_host = true;

// 릴레이 예약 확보
self.p2p_node.start().await?;

Ok(())
}

async fn start_connection_monitoring(&self) {
let connections = self.player_connections.clone();
tokio::spawn(async move {
let mut interval = tokio::time::interval(Duration::from_secs(1));
loop {
interval.tick().await;
for (peer_id, connection) in &connections {
// 지연시간 측정
```

```
if connection.last_seen.elapsed() > Duration::from_secs(5) {
warn!("Player {} seems disconnected", peer_id);
}
// 연결 품질 확인
match connection.connection_type {
ConnectionType::Relayed => {
info!("Player {} using relay (latency: {:?})", peer_id, connection.latency);
}
ConnectionType::Direct => {
info!("Player {} direct connection (latency: {:?})", peer_id, connection.latency
}
ConnectionType::Upgrading => {
info!("Player {} connection upgrading via DCUtR", peer_id);
}
}
}
}
});
}


pub async fn run_game_loop(&mut self) -> Result<(), Box<dyn std::error::Error>>
loop {
tokio::select! {
```

```
// P2P 네트워크 이벤트 처리
_ = self.p2p_node.run() => {
warn!("P2P node stopped");
break;
}
// 게임 메시지 처리
Some(message) = self.message_receiver.recv() => {
self.handle_game_message(message).await;
}
// 게임 로직 업데이트
_ = tokio::time::sleep(Duration::from_millis(16)) => { // 60 FPS
if self.game_state.game_started {
self.update_game_state().await;
}
}
}
}
Ok(())
}

async fn handle_game_message(&mut self, message: GameMessage) {
match message {
GameMessage::PlayerJoined(peer_id) => {
```

```
info!("Player joined: {}", peer_id);
self.game_state.players.push(peer_id);
// 새 플레이어와 연결 설정
if let Err(e) = self.p2p_node.connect_to_peer(peer_id, vec![]).await {
warn!("Failed to connect to new player {}: {}", peer_id, e);
}
}
GameMessage::PlayerLeft(peer_id) => {
info!("Player left: {}", peer_id);
self.game_state.players.retain(|p| *p != peer_id);
self.player_connections.remove(&peer_id);
}
GameMessage::GameData(data) => {
// 게임 데이터 처리
self.process_game_data(data).await;
}
GameMessage::LatencyTest(timestamp) => {
let latency = timestamp.elapsed();
debug!("Latency measurement: {:?}", latency);
}
}
}
```

```
async fn update_game_state(&mut self) {
// 게임 상태 업데이트 로직
// 모든 연결된 플레이어에게 상태 전송
for peer_id in &self.game_state.players {
if let Some(connection) = self.player_connections.get(peer_id) {
// 연결 타입에 따른 최적화된 데이터 전송
match connection.connection_type {
ConnectionType::Direct => {
// 직접 연결: 고품질 데이터 전송
self.send_high_quality_update(*peer_id).await;
}
ConnectionType::Relayed => {
// 릴레이 연결: 압축된 데이터 전송
self.send_compressed_update(*peer_id).await;
}
ConnectionType::Upgrading => {
// 업그레이드 중: 최소한의 데이터만 전송
self.send_minimal_update(*peer_id).await;
}
}
}
}
}
```

```
async fn send_high_quality_update(&self, peer_id: PeerId) {
// 고품질 게임 상태 업데이트
}

async fn send_compressed_update(&self, peer_id: PeerId) {
// 압축된 게임 상태 업데이트
}

async fn send_minimal_update(&self, peer_id: PeerId) {
// 최소한의 게임 상태 업데이트
}

async fn process_game_data(&mut self, data: Vec<u8>) {
// 게임 데이터 처리 로직
}
}

// 게임 서버 (매치메이킹)
pub struct GameMatchmakingServer {
p2p_node: AdvancedP2PNode,
active_rooms: HashMap<String, GameRoom>,
waiting_players: Vec<PeerId>,
```

```
}

#[derive(Debug)]
pub struct GameRoom {
id: String,
host: PeerId,
players: Vec<PeerId>,
max_players: usize,
created_at: std::time::Instant,
}

impl GameMatchmakingServer {
pub async fn new() -> Result<Self, Box<dyn std::error::Error>> {
let config = NodeConfig {
enable_relay_server: true, // 매치메이킹 서버는 릴레이 역할도 수행
relay_servers: vec![], // 자체가 릴레이 서버
aggressive_hole_punching: false,
connection_timeout: Duration::from_secs(30),
autonat_config: autonat::Config {
only_global_ips: true, // 서버는 공개 IP 필요
..Default::default()
},
};
```

```
let p2p_node = AdvancedP2PNode::new(config).await?;


Ok(Self {
p2p_node,
active_rooms: HashMap::new(),
waiting_players: Vec::new(),
})
}


pub async fn start_server(&mut self) -> Result<(), Box<dyn std::error::Error>> {
info!("Starting game matchmaking server");
self.p2p_node.start().await?;
// 매치메이킹 로직 시작
self.start_matchmaking_loop().await;
Ok(())
}


async fn start_matchmaking_loop(&self) {
tokio::spawn(async move {
let mut interval = tokio::time::interval(Duration::from_secs(5));
loop {
interval.tick().await;
```

```
// 대기 중인 플레이어들을 매칭

// 게임 룸 생성 및 관리

}

});

}

}
```

2. 분산 파일 저장 시스템

아키텍처

```
┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐

| Storage Node | | Bootstrap | | Storage Node |

| (Home User) |◄──►| Relay Server |◄──►| (Office) |

| | | | | |

└─────────────────┘  └─────────────────┘  └─────────────────┘

| | |

└──────────────────────┼──────────────────────┘

|

┌─────────────────┐

| Metadata |

| Coordinator |

└─────────────────┘
```

구현 예제

```rust
// distributed_storage.rs
use libp2p::{PeerId, Multiaddr};
use std::{collections::HashMap, path::PathBuf};
use tokio::fs;

pub struct DistributedStorageNode {
p2p_node: AdvancedP2PNode,
storage_config: StorageConfig,
file_index: HashMap<String, FileMetadata>,
peer_storage_info: HashMap<PeerId, PeerStorageInfo>,
}

#[derive(Debug, Clone)]
pub struct StorageConfig {
storage_path: PathBuf,
max_storage_size: u64,
replication_factor: usize,
chunk_size: usize,
}

#[derive(Debug, Clone)]
```

```rust
pub struct FileMetadata {
file_id: String,
file_name: String,
file_size: u64,
chunk_hashes: Vec<String>,
stored_peers: Vec<PeerId>,
created_at: std::time::SystemTime,
}

#[derive(Debug)]
pub struct PeerStorageInfo {
available_space: u64,
stored_files: Vec<String>,
connection_quality: ConnectionQuality,
last_seen: std::time::Instant,
}

#[derive(Debug, Clone)]
pub enum ConnectionQuality {
Excellent, // Direct connection, low latency
Good, // Direct connection, medium latency
Fair, // Relay connection, stable
Poor, // Relay connection, unstable
```

```
}

impl DistributedStorageNode {
pub async fn new(storage_config: StorageConfig, bootstrap_relays: Vec<(PeerId, M
let config = NodeConfig {
enable_relay_server: false,
relay_servers: bootstrap_relays,
aggressive_hole_punching: false, // 안정성 우선
connection_timeout: Duration::from_secs(60),
autonat_config: autonat::Config {
retry_interval: Duration::from_secs(90),
refresh_interval: Duration::from_secs(900),
confidence_max: 5, // 높은 신뢰도 요구
..Default::default()
},
};

let p2p_node = AdvancedP2PNode::new(config).await?;

// 저장소 디렉토리 생성
fs::create_dir_all(&storage_config.storage_path).await?;

Ok(Self {
```

```
p2p_node,
storage_config,
file_index: HashMap::new(),
peer_storage_info: HashMap::new(),
})
}

pub async fn store_file(&mut self, file_path: PathBuf) -> Result<String, Box<dyn
info!("Storing file: {:?}", file_path);

// 파일 읽기 및 청킹
let file_data = fs::read(&file_path).await?;
let chunks = self.create_chunks(&file_data);
let file_id = self.generate_file_id(&file_data);

// 최적의 저장 피어 선택
let storage_peers = self.select_storage_peers(chunks.len()).await?;

// 청크를 선택된 피어들에 분산 저장
let mut stored_chunks = Vec::new();
for (i, chunk) in chunks.iter().enumerate() {
let peer_id = storage_peers[i % storage_peers.len()];
match self.store_chunk_to_peer(peer_id, &file_id, i, chunk).await {
```

```
Ok(chunk_hash) => {
stored_chunks.push(chunk_hash);
info!("Chunk {} stored to peer {}", i, peer_id);
}
Err(e) => {
warn!("Failed to store chunk {} to peer {}: {}", i, peer_id, e);
// 다른 피어에 재시도
if let Some(backup_peer) = self.find_backup_peer(&storage_peers).await {
match self.store_chunk_to_peer(backup_peer, &file_id, i, chunk).await {
Ok(chunk_hash) => stored_chunks.push(chunk_hash),
Err(e2) => return Err(format!("Failed to store chunk {}: {}", i, e2).into()),
}
}
}
}
}


// 파일 메타데이터 저장
let metadata = FileMetadata {
file_id: file_id.clone(),
file_name: file_path.file_name().unwrap().to_string_lossy().to_string(),
file_size: file_data.len() as u64,
chunk_hashes: stored_chunks,
```

```
stored_peers: storage_peers,

created_at: std::time::SystemTime::now(),

};


self.file_index.insert(file_id.clone(), metadata);


info!("File stored successfully with ID: {}", file_id);

Ok(file_id)

}


pub async fn retrieve_file(&mut self, file_id: &str, output_path: PathBuf) -> Re

info!("Retrieving file: {}", file_id);


let metadata = self.file_index.get(file_id)

.ok_or("File not found")?;


// 청크들을 병렬로 다운로드

let mut chunk_futures = Vec::new();

for (i, chunk_hash) in metadata.chunk_hashes.iter().enumerate() {

let peer_candidates = self.find_chunk_peers(file_id, i).await;

let chunk_future = self.download_chunk_from_peers(file_id, i, chunk_hash, peer_c

chunk_futures.push(chunk_future);

}
```

```
// 모든 청크 다운로드 완료 대기
let chunks = futures::future::try_join_all(chunk_futures).await?;

// 청크들을 합쳐서 원본 파일 복원
let file_data = self.reconstruct_file(chunks);
fs::write(output_path, file_data).await?;

info!("File retrieved successfully: {}", file_id);
Ok(())
}

async fn select_storage_peers(&self, chunk_count: usize) -> Result<Vec<PeerId>,
let mut suitable_peers = Vec::new();

for (peer_id, storage_info) in &self.peer_storage_info {
// 연결 품질과 저장 공간을 고려하여 피어 선택
match storage_info.connection_quality {
ConnectionQuality::Excellent | ConnectionQuality::Good => {
if storage_info.available_space > self.storage_config.chunk_size as u64 {
suitable_peers.push((*peer_id, 100)); // 높은 우선순위
}
}
```

```
ConnectionQuality::Fair => {
if storage_info.available_space > self.storage_config.chunk_size as u64 {
suitable_peers.push((*peer_id, 50)); // 중간 우선순위
}
}
ConnectionQuality::Poor => {
// 낮은 품질 연결은 백업용으로만 사용
suitable_peers.push((*peer_id, 10));
}
}
}

// 우선순위에 따라 정렬
suitable_peers.sort_by(|a, b| b.1.cmp(&a.1));

// 필요한 수만큼 피어 선택 (복제 팩터 고려)
let needed_peers = chunk_count * self.storage_config.replication_factor;
let selected_peers: Vec<PeerId> = suitable_peers
.into_iter()
.take(needed_peers)
.map(|(peer_id, _)| peer_id)
.collect();
```

```
if selected_peers.len() < needed_peers {
return Err("Not enough suitable peers for storage".into());
}

Ok(selected_peers)
}

async fn store_chunk_to_peer(&mut self, peer_id: PeerId, file_id: &str, chunk_in
// 피어에 연결 (필요시)
if !self.p2p_node.is_connected(&peer_id) {
self.p2p_node.connect_to_peer(peer_id, vec![]).await?;
}

// 청크 해시 계산
let chunk_hash = self.calculate_hash(chunk_data);

// 청크 데이터 전송 (실제 구현에서는 커스텀 프로토콜 사용)
// 여기서는 개념적 구현만 표시
info!("Sending chunk {} of file {} to peer {}", chunk_index, file_id, peer_id);

// 전송 성공 시 피어 정보 업데이트
if let Some(storage_info) = self.peer_storage_info.get_mut(&peer_id) {
storage_info.stored_files.push(format!("{}:{}", file_id, chunk_index));
```

```
storage_info.available_space = storage_info.available_space.saturating_sub(chunk

}


Ok(chunk_hash)

}


async fn download_chunk_from_peers(&mut self, file_id: &str, chunk_index: usize,

for peer_id in peer_candidates {

match self.download_chunk_from_peer(peer_id, file_id, chunk_index).await {

Ok(chunk_data) => {

// 해시 검증

let actual_hash = self.calculate_hash(&chunk_data);

if actual_hash == expected_hash {

return Ok(chunk_data);

} else {

warn!("Hash mismatch for chunk {} from peer {}", chunk_index, peer_id);

}

}

Err(e) => {

warn!("Failed to download chunk {} from peer {}: {}", chunk_index, peer_id, e);

}

}

}
```

```
Err(format!("Failed to download chunk {} from any peer", chunk_index).into())
}

async fn download_chunk_from_peer(&mut self, peer_id: PeerId, file_id: &str, chu
// 피어에 연결 (필요시)
if !self.p2p_node.is_connected(&peer_id) {
self.p2p_node.connect_to_peer(peer_id, vec![]).await?;
}

// 청크 요청 및 다운로드 (실제 구현에서는 커스텀 프로토콜 사용)
info!("Downloading chunk {} of file {} from peer {}", chunk_index, file_id, peer

// 모의 데이터 반환 (실제로는 네트워크에서 받음)
Ok(vec![0u8; self.storage_config.chunk_size])
}

fn create_chunks(&self, data: &[u8]) -> Vec<Vec<u8>> {
data.chunks(self.storage_config.chunk_size)
.map(|chunk| chunk.to_vec())
.collect()
}
```

```
fn reconstruct_file(&self, chunks: Vec<Vec<u8>>) -> Vec<u8> {
chunks.into_iter().flatten().collect()
}

fn generate_file_id(&self, data: &[u8]) -> String {
// 실제로는 SHA-256 등의 해시 사용
format!("file_{}", data.len())
}

fn calculate_hash(&self, data: &[u8]) -> String {
// 실제로는 SHA-256 등의 해시 사용
format!("hash_{}", data.len())
}

async fn find_chunk_peers(&self, file_id: &str, chunk_index: usize) -> Vec<PeerI
let chunk_key = format!("{}:{}", file_id, chunk_index);
self.peer_storage_info
.iter()
.filter(|(_, info)| info.stored_files.contains(&chunk_key))
.map(|(peer_id, _)| *peer_id)
.collect()
}
```

```
async fn find_backup_peer(&self, excluded_peers: &[PeerId]) -> Option<PeerId> {
self.peer_storage_info
.keys()
.find(|peer_id| !excluded_peers.contains(peer_id))
.copied()
}

pub async fn run_storage_node(&mut self) -> Result<(), Box<dyn std::error::Error
info!("Starting distributed storage node");

// 피어 발견 및 연결 관리 시작
self.start_peer_discovery().await;

// 저장소 상태 모니터링 시작
self.start_storage_monitoring().await;

// 메인 이벤트 루프
self.p2p_node.run().await
}

async fn start_peer_discovery(&self) {
tokio::spawn(async move {
let mut interval = tokio::time::interval(Duration::from_secs(30));
```

```
loop {
interval.tick().await;
// 새로운 저장소 피어 발견 및 연결
info!("Discovering storage peers...");
}
});
}

async fn start_storage_monitoring(&self) {
tokio::spawn(async move {
let mut interval = tokio::time::interval(Duration::from_secs(60));
loop {
interval.tick().await;
// 저장된 파일 무결성 검사
// 피어 연결 상태 확인
// 저장 공간 사용량 모니터링
info!("Monitoring storage health...");
}
});
}
}
```

3. IoT 디바이스 네트워크

아키텍처

```
┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐
│ IoT Gateway │  │ Cloud Relay │  │ IoT Device │
│ (Edge Router) │◄──►│ Server │◄──►│ (Sensor) │
│ │  │  │  │ NAT 뒤 │
└─────────────────┘  └─────────────────┘  └─────────────────┘
│  │  │
└──────────────────────┼──────────────────────┘
│
┌─────────────────┐
│ Management │
│ Dashboard │
└─────────────────┘
```

구현 예제

```rust
// iot_network.rs
use libp2p::{PeerId, Multiaddr};
use serde::{Deserialize, Serialize};
use std::{collections::HashMap, time::Duration};
use tokio::time;

pub struct IoTNetworkNode {
```

```
p2p_node: AdvancedP2PNode,
device_type: DeviceType,
sensor_data: HashMap<String, SensorReading>,
device_registry: HashMap<PeerId, DeviceInfo>,
}

#[derive(Debug, Clone)]
pub enum DeviceType {
Gateway, // 게이트웨이 (릴레이 서버 역할)
Sensor, // 센서 디바이스
Actuator, // 액추에이터 디바이스
Controller, // 컨트롤러
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct SensorReading {
sensor_id: String,
value: f64,
unit: String,
timestamp: u64,
device_id: PeerId,
}
```

```rust
#[derive(Debug, Clone)]
pub struct DeviceInfo {
device_type: DeviceType,
capabilities: Vec<String>,
last_seen: std::time::Instant,
battery_level: Option<u8>, // 배터리 레벨 (0-100)
connection_quality: ConnectionQuality,
}


impl IoTNetworkNode {
pub async fn new_iot_gateway(cloud_relays: Vec<(PeerId, Multiaddr)>) -> Result<S
let config = NodeConfig {
enable_relay_server: true, // 게이트웨이는 릴레이 서버 역할
relay_servers: cloud_relays,
aggressive_hole_punching: false, // 안정성 우선
connection_timeout: Duration::from_secs(30),
autonat_config: autonat::Config {
retry_interval: Duration::from_secs(120),
refresh_interval: Duration::from_secs(1800), // 30분
confidence_max: 3,
..Default::default()
},
};
```

```
let p2p_node = AdvancedP2PNode::new(config).await?;


Ok(Self {
p2p_node,
device_type: DeviceType::Gateway,
sensor_data: HashMap::new(),
device_registry: HashMap::new(),
})
}


pub async fn new_iot_device(device_type: DeviceType, gateway_relays: Vec<(PeerId
let config = NodeConfig {
enable_relay_server: false,
relay_servers: gateway_relays,
aggressive_hole_punching: false, // 배터리 절약
connection_timeout: Duration::from_secs(60),
autonat_config: autonat::Config {
retry_interval: Duration::from_secs(300), // 5분 - 배터리 절약
refresh_interval: Duration::from_secs(3600), // 1시간
confidence_max: 2,
..Default::default()
},
```

```
};

let p2p_node = AdvancedP2PNode::new(config).await?;

Ok(Self {
p2p_node,
device_type,
sensor_data: HashMap::new(),
device_registry: HashMap::new(),
})
}

pub async fn start_iot_node(&mut self) -> Result<(), Box<dyn std::error::Error>>
info!("Starting IoT node as {:?}", self.device_type);

self.p2p_node.start().await?;

match self.device_type {
DeviceType::Gateway => {
self.start_gateway_services().await;
}
DeviceType::Sensor => {
self.start_sensor_services().await;
```

```
}
DeviceType::Actuator => {
self.start_actuator_services().await;
}
DeviceType::Controller => {
self.start_controller_services().await;
}
}

Ok(())
}

async fn start_gateway_services(&self) {
info!("Starting gateway services");

// 디바이스 발견 서비스
tokio::spawn(async move {
let mut interval = time::interval(Duration::from_secs(60));
loop {
interval.tick().await;
info!("Discovering IoT devices...");
// 새로운 IoT 디바이스 검색 및 등록
}
```

```
});

// 데이터 수집 서비스
tokio::spawn(async move {
let mut interval = time::interval(Duration::from_secs(30));
loop {
interval.tick().await;
info!("Collecting sensor data...");
// 연결된 센서들로부터 데이터 수집
}
});

// 클라우드 동기화 서비스
tokio::spawn(async move {
let mut interval = time::interval(Duration::from_secs(300)); // 5분
loop {
interval.tick().await;
info!("Syncing with cloud...");
// 수집된 데이터를 클라우드로 전송
}
});
}
```

```
async fn start_sensor_services(&self) {
info!("Starting sensor services");

// 센서 데이터 생성 및 전송
tokio::spawn(async move {
let mut interval = time::interval(Duration::from_secs(60)); // 1분마다
loop {
interval.tick().await;
// 센서 데이터 생성 (모의)
let reading = SensorReading {
sensor_id: "temp_01".to_string(),
value: 23.5 + (rand::random::<f64>() - 0.5) * 5.0, // 20-27도 범위
unit: "°C".to_string(),
timestamp: std::time::SystemTime::now()
.duration_since(std::time::UNIX_EPOCH)
.unwrap()
.as_secs(),
device_id: PeerId::random(), // 실제로는 자신의 peer_id 사용
};

info!("Generated sensor reading: {:?}", reading);
// 게이트웨이로 데이터 전송
}
```

```
});

// 배터리 상태 모니터링
tokio::spawn(async move {
let mut interval = time::interval(Duration::from_secs(600)); // 10분마다
loop {
interval.tick().await;
// 배터리 레벨 확인 (모의)
let battery_level = 85u8; // 실제로는 시스템에서 읽음
if battery_level < 20 {
warn!("Low battery: {}%", battery_level);
// 절전 모드 활성화
} else if battery_level < 50 {
info!("Battery level: {}%", battery_level);
// 데이터 전송 빈도 감소
}
}
});
}

async fn start_actuator_services(&self) {
info!("Starting actuator services");
```

```
// 명령 수신 및 실행
tokio::spawn(async move {
loop {
// 게이트웨이나 컨트롤러로부터 명령 대기
info!("Waiting for actuator commands...");
time::sleep(Duration::from_secs(10)).await;
}
});
}

async fn start_controller_services(&self) {
info!("Starting controller services");

// 자동화 규칙 실행
tokio::spawn(async move {
let mut interval = time::interval(Duration::from_secs(30));
loop {
interval.tick().await;
// 센서 데이터 기반 자동화 규칙 실행
info!("Executing automation rules...");
// 예: 온도가 25도 이상이면 에어컨 켜기
}
});
```

```
}

pub async fn send_sensor_data(&mut self, reading: SensorReading) -> Result<(), B
info!("Sending sensor data: {:?}", reading);

// 게이트웨이 찾기
let gateway_peers: Vec<PeerId> = self.device_registry
.iter()
.filter(|(_, info)| matches!(info.device_type, DeviceType::Gateway))
.map(|(peer_id, _)| *peer_id)
.collect();

if gateway_peers.is_empty() {
return Err("No gateway found".into());
}

// 가장 좋은 연결 품질을 가진 게이트웨이 선택
let best_gateway = gateway_peers
.iter()
.min_by_key(|peer_id| {
match self.device_registry.get(peer_id).unwrap().connection_quality {
ConnectionQuality::Excellent => 0,
ConnectionQuality::Good => 1,
```

```
ConnectionQuality::Fair => 2,
ConnectionQuality::Poor => 3,
}
})
.unwrap();

// 데이터 전송 (실제 구현에서는 커스텀 프로토콜 사용)
if !self.p2p_node.is_connected(best_gateway) {
self.p2p_node.connect_to_peer(*best_gateway, vec![]).await?;
}

// 센서 데이터 저장
self.sensor_data.insert(reading.sensor_id.clone(), reading);

Ok(())
}

pub async fn execute_actuator_command(&mut self, target_device: PeerId, command:
info!("Executing actuator command: {:?} on device {}", command, target_device);

// 대상 디바이스에 연결
if !self.p2p_node.is_connected(&target_device) {
self.p2p_node.connect_to_peer(target_device, vec![]).await?;
```

```
}

// 명령 전송 (실제 구현에서는 커스텀 프로토콜 사용)
info!("Command sent to device {}", target_device);

Ok(())
}

pub async fn run_iot_network(&mut self) -> Result<(), Box<dyn std::error::Error>
// 디바이스 상태 모니터링 시작
self.start_device_monitoring().await;

// 메인 이벤트 루프
self.p2p_node.run().await
}

async fn start_device_monitoring(&self) {
tokio::spawn(async move {
let mut interval = time::interval(Duration::from_secs(120)); // 2분마다
loop {
interval.tick().await;
info!("Monitoring device health...");
// 연결된 디바이스들의 상태 확인
```

```
// 오프라인 디바이스 감지
// 배터리 부족 디바이스 알림
}
});
}
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum ActuatorCommand {
TurnOn,
TurnOff,
SetValue(f64),
SetMode(String),
}

// IoT 네트워크 관리자
pub struct IoTNetworkManager {
gateways: Vec<IoTNetworkNode>,
devices: Vec<IoTNetworkNode>,
network_topology: HashMap<PeerId, Vec<PeerId>>, // 연결 관계
}

impl IoTNetworkManager {
```

```
pub async fn new() -> Self {
Self {
gateways: Vec::new(),
devices: Vec::new(),
network_topology: HashMap::new(),
}
}


pub async fn deploy_network(&mut self, config: IoTNetworkConfig) -> Result<(), B
info!("Deploying IoT network with {} gateways and {} devices",
config.gateway_count, config.device_count);


// 게이트웨이 배포
for i in 0..config.gateway_count {
let gateway = IoTNetworkNode::new_iot_gateway(config.cloud_relays.clone()).await
self.gateways.push(gateway);
info!("Gateway {} deployed", i);
}


// 디바이스 배포
for i in 0..config.device_count {
let device_type = if i % 3 == 0 {
DeviceType::Sensor
```

```
} else if i % 3 == 1 {
DeviceType::Actuator
} else {
DeviceType::Controller
};

let device = IoTNetworkNode::new_iot_device(device_type, config.gateway_relays.c
self.devices.push(device);
info!("Device {} deployed as {:?}", i, device_type);
}

// 네트워크 시작
self.start_all_nodes().await?;

Ok(())
}

async fn start_all_nodes(&mut self) -> Result<(), Box<dyn std::error::Error>> {
// 게이트웨이 시작
for gateway in &mut self.gateways {
gateway.start_iot_node().await?;
}
```

```
// 디바이스 시작 (지연을 두어 순차적으로)
for (i, device) in self.devices.iter_mut().enumerate() {
tokio::time::sleep(Duration::from_secs(2)).await; // 2초 지연
device.start_iot_node().await?;
info!("Device {} started", i);
}

Ok(())
}
}

#[derive(Debug, Clone)]
pub struct IoTNetworkConfig {
pub gateway_count: usize,
pub device_count: usize,
pub cloud_relays: Vec<(PeerId, Multiaddr)>,
pub gateway_relays: Vec<(PeerId, Multiaddr)>,
}
```

배포 스크립트

Docker Compose 배포

```yaml
# docker-compose.yml
version: '3.8'

services:
# 게임 릴레이 서버
game-relay:
build:
context: .
dockerfile: Dockerfile.game-relay
ports:
- "4001:4001"
- "8080:8080"
environment:
- RUST_LOG=info,libp2p=debug
- RELAY_TYPE=game
networks:
- p2p-network
restart: unless-stopped

# 스토리지 릴레이 서버
storage-relay:
build:
```

```yaml
    context: .
    dockerfile: Dockerfile.storage-relay
    ports:
    - "4002:4001"
    - "8081:8080"
    environment:
    - RUST_LOG=info,libp2p=debug
    - RELAY_TYPE=storage
    volumes:
    - storage-data:/data
    networks:
    - p2p-network
    restart: unless-stopped


  # IoT 게이트웨이
  iot-gateway:
    build:
    context: .
    dockerfile: Dockerfile.iot-gateway
    ports:
    - "4003:4001"
    - "8082:8080"
    environment:
```

```yaml
- RUST_LOG=info,libp2p=debug
- DEVICE_TYPE=gateway
networks:
- p2p-network
restart: unless-stopped


# 모니터링 (Prometheus)
prometheus:
image: prom/prometheus:latest
ports:
- "9090:9090"
volumes:
- ./prometheus.yml:/etc/prometheus/prometheus.yml
- prometheus-data:/prometheus
networks:
- p2p-network


# 대시보드 (Grafana)
grafana:
image: grafana/grafana:latest
ports:
- "3000:3000"
environment:
```

```yaml
    - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
    - grafana-data:/var/lib/grafana
    - ./grafana/dashboards:/etc/grafana/provisioning/dashboards
    - ./grafana/datasources:/etc/grafana/provisioning/datasources
    networks:
    - p2p-network

volumes:
  storage-data:
  prometheus-data:
  grafana-data:

networks:
  p2p-network:
    driver: bridge
```

Kubernetes 배포

```yaml
# k8s-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
```

```yaml
name: p2p-relay-cluster

spec:

replicas: 3

selector:

matchLabels:

app: p2p-relay

template:

metadata:

labels:

app: p2p-relay

spec:

containers:

- name: relay-server

image: advanced-p2p:latest

ports:

- containerPort: 4001

- containerPort: 8080

env:

- name: RUST_LOG

value: "info,libp2p=debug"

- name: ENABLE_RELAY_SERVER

value: "true"

resources:
```

```yaml
requests:
memory: "512Mi"
cpu: "500m"
limits:
memory: "1Gi"
cpu: "1000m"
livenessProbe:
httpGet:
path: /health
port: 8080
initialDelaySeconds: 30
periodSeconds: 10
readinessProbe:
httpGet:
path: /ready
port: 8080
initialDelaySeconds: 5
periodSeconds: 5


---
apiVersion: v1
kind: Service
metadata:
```

```yaml
name: p2p-relay-service

spec:

selector:

app: p2p-relay

ports:

- name: p2p

port: 4001

targetPort: 4001

- name: http

port: 8080

targetPort: 8080

type: LoadBalancer



---

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: p2p-monitoring-ingress

spec:

rules:

- host: p2p-monitor.example.com

http:

paths:
```

```yaml
- path: /
pathType: Prefix
backend:
service:
name: p2p-relay-service
port:
number: 8080
```

이러한 배포 예제들을 통해 다양한 환경에서 고급 P2P 기능을 활용한 애플리케이션을 구축하고 운영할 수 있습니다. 각 시나리오는 특정 요구사항에 맞게 최적화되어 있으며, 실제 프로덕션 환경에서 사용할 수 있는 수준의 구현을 제공합니다.